

+ 26

Death by a Thousand Tape Entries

Low-Overhead Automatic Differentiation for
Scientific and Financial C++

JORG LOTZE



20
26



Jorg Lotze

Xcelerit Technical Lead

- Over 25 years in building high-performance C++ systems for global investment banks and scientific computing
- Creator and maintainer of XAD automatic differentiation tool

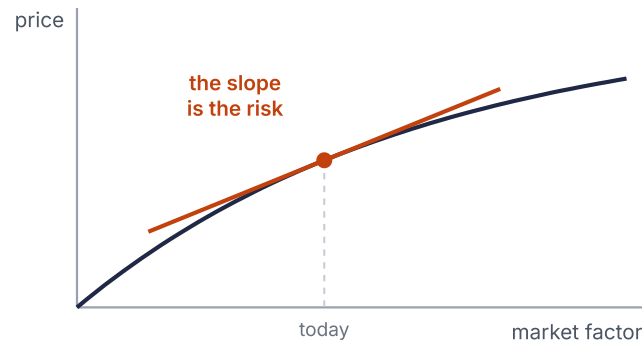


Today's talk

- What adjoint AD is, and why finance needs it fast
- Ten design rules that take a tape from 55× to 3.9×
- What that looks like inside a 300 kLOC quant library

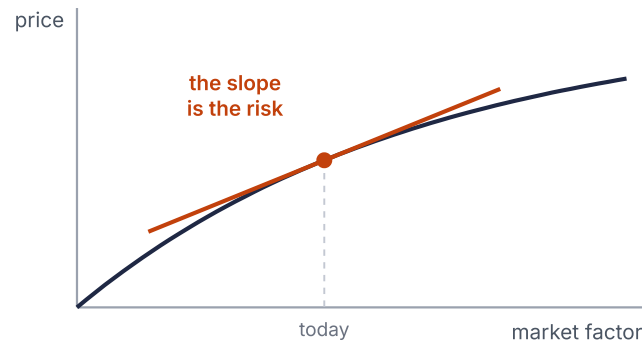
What a sensitivity is

- A price depends on the market: curves, spreads, vols
- A **sensitivity** is how much it moves when one factor moves
- On an option book they are the **Greeks**:
delta, vega, and the rest
- Traders hedge on them, regulators require them
- Machine learning calls the same thing a **gradient**



What a sensitivity is

- A price depends on the market: curves, spreads, vols
- A **sensitivity** is how much it moves when one factor moves
- On an option book they are the **Greeks**:
delta, vega, and the rest
- Traders hedge on them, regulators require them
- Machine learning calls the same thing a **gradient**

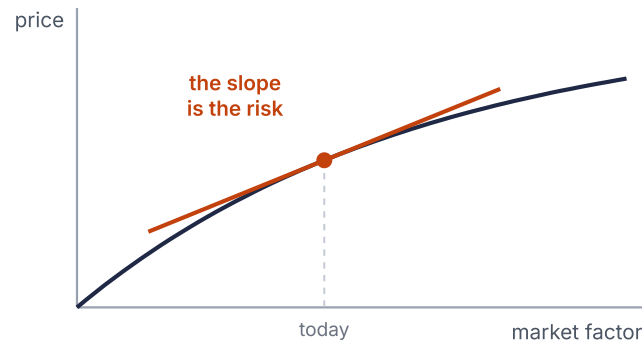


Example: A portfolio risk run of **30 minutes**, and **1,000** market risk factors to report:



What a sensitivity is

- A price depends on the market: curves, spreads, vols
- A **sensitivity** is how much it moves when one factor moves
- On an option book they are the **Greeks**:
delta, vega, and the rest
- Traders hedge on them, regulators require them
- Machine learning calls the same thing a **gradient**



Example: A portfolio risk run of **30 minutes**, and **1,000** market risk factors to report:



Two ways to get 40 derivatives

The same CVA valuation underneath each.

The value (primal)

```
// 40 market inputs in, one number out
double cva(const double m[40]);
```

Bump and revalue (finite differences)

```
for (int i = 0; i < 40; ++i) {
    m[i] += h;
    g[i] = (cva(m) - base) / h;
    m[i] -= h;
}
```

41 valuations, so **41 × primal**

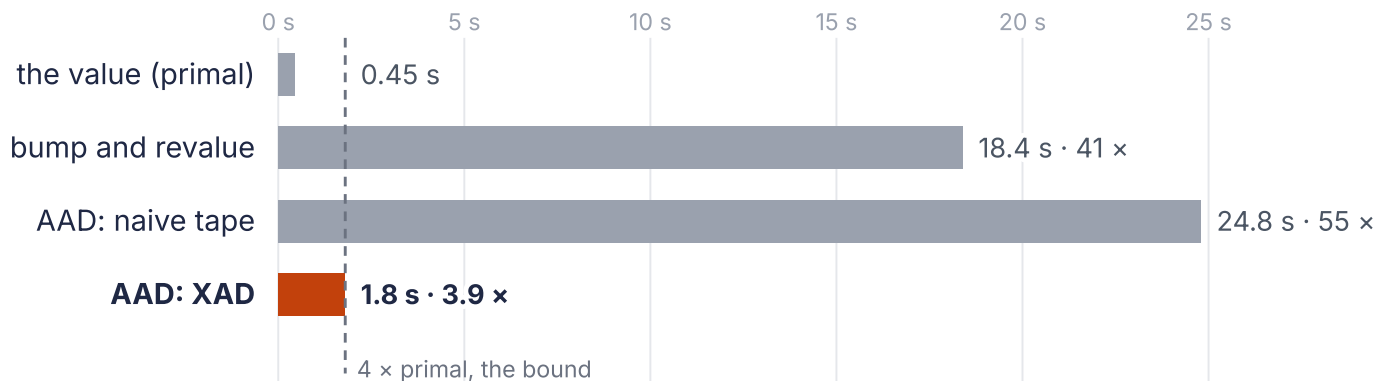
Record and chain rule (adjoint algorithmic differentiation)

```
std::copy_n(m, 40, m_ad); // active copy
Real y = cva_aad(m_ad);   // same valuation
derivative(y) = 1.0;
tape.computeAdjoint();    // all 40 at once
for (int i = 0; i < 40; ++i)
    g[i] = derivative(m_ad[i]);
```

Arithmetically **4 × primal** at most, however many inputs

What they actually cost

CVA, 10,000 Monte-Carlo paths, 40 market inputs

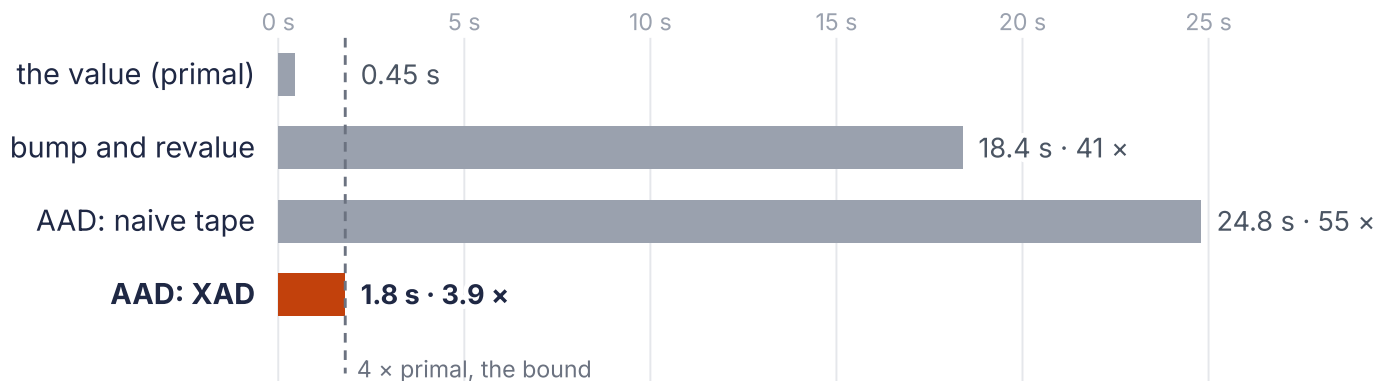


AAD with naive tape misses theoretical by **fourteen times**, worse than bump-and-revalue!

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · median of 10 runs after 3 warm-ups

What they actually cost

CVA, 10,000 Monte-Carlo paths, 40 market inputs



AAD with naive tape misses theoretical by **fourteen times**, worse than bump-and-revalue!

The rest of this talk is how a tape gets from 55x to 3.9x.

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · median of 10 runs after 3 warm-ups

How AAD reached finance, slowly

The algorithm

- **1970** Linnainmaa's thesis writes down the reverse mode
- **1980** Speelpenning: the first *automatic* implementation
- **1983** Baur and Strassen: a gradient costs a constant times the function, whatever the number of inputs
- **1986** Rumelhart, Hinton and Williams publish it as **backpropagation**
- **2006** Giles and Glasserman, *Smoking adjoints*: every Greek of a Monte Carlo pricer for **less than 4×** the pricing run

The obligation

- **2008** the crisis. Risk that has to be produced every night.
- **2016** FRTB: capital under the sensitivities-based method
- **2016** ISDA SIMM: initial margin on uncleared trades based on sensitivities.
- **2020** SA-CVA: capital charge based on CVA sensitivities
- **2022** the last margin phase: initial margin for firms down to €8bn notional, in practice computed with SIMM

How AAD reached finance, slowly

The algorithm

- **1970** Linnainmaa's thesis writes down the reverse mode
- **1980** Speelpenning: the first *automatic* implementation
- **1983** Baur and Strassen: a gradient costs a constant times the function, whatever the number of inputs
- **1986** Rumelhart, Hinton and Williams publish it as **backpropagation**
- **2006** Giles and Glasserman, *Smoking adjoints*: every Greek of a Monte Carlo pricer for **less than 4×** the pricing run

The mathematics has been settled since 1983.

The hard part is a quant library of hundreds of thousands of lines that has to finish overnight.

The obligation

- **2008** the crisis. Risk that has to be produced every night.
- **2016** FRTB: capital under the sensitivities-based method
- **2016** ISDA SIMM: initial margin on uncleared trades based on sensitivities.
- **2020** SA-CVA: capital charge based on CVA sensitivities
- **2022** the last margin phase: initial margin for firms down to €8bn notional, in practice computed with SIMM

1. Reverse AD and the naive tape

What the obvious implementation costs, and where it goes.

The running example: d_1 term in Black-Scholes

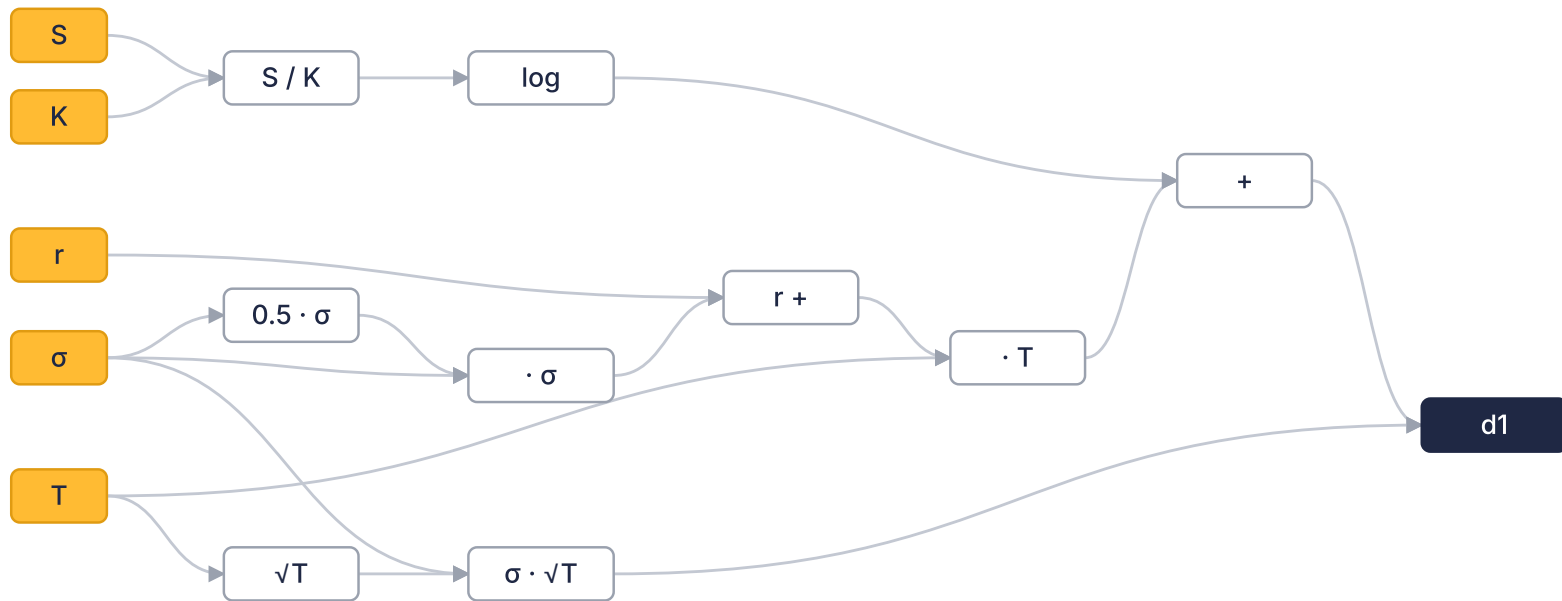
```
template <class T>
T d1(const T& S, const T& K, const T& r, const T& sigma, const T& T_)
{
    using std::log, std::sqrt;
    return (log(S / K) + (r + 0.5 * sigma * sigma) * T_) / (sigma * sqrt(T_));
}
```

- 5 active inputs
- 2 divisions
- 3 products, 1 active×passive product `0.5 * sigma`
- 2 additions
- 2 unary math functions (`log` , `sqrt`)

→ 10 operations

Graph of ten operations to record to tape

$$d_1 = \frac{\log(S/K) + (r + \sigma^2/2) T}{\sigma \sqrt{T}}$$



Backward sweep: chain rule

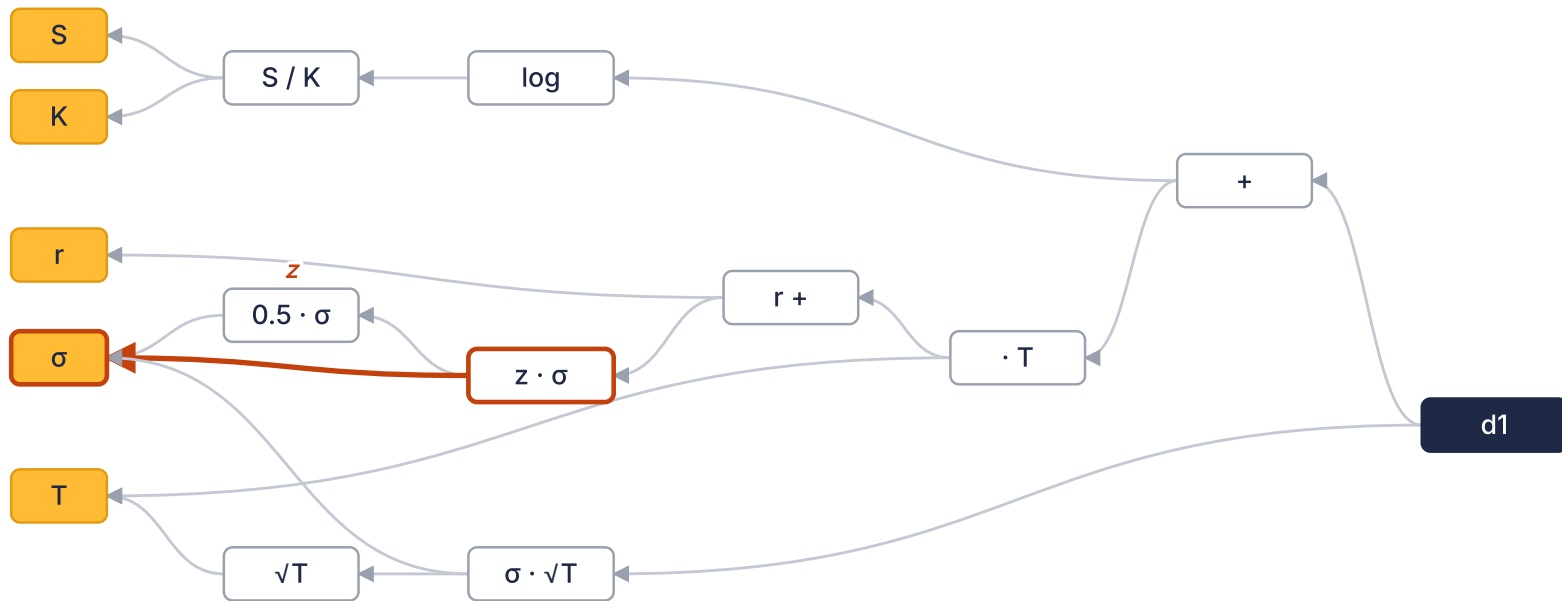
$$\bar{v} \equiv \frac{\partial d_1}{\partial v} \qquad \overline{d_1} = 1 \qquad \bar{x} += \frac{\partial y}{\partial x} \bar{y}$$

the adjoint of every value, seeded at the result, pushed back one edge at a time

Backward sweep: chain rule

$$\bar{v} \equiv \frac{\partial d_1}{\partial v} \quad \overline{d_1} = 1 \quad \bar{x} += \frac{\partial y}{\partial x} \bar{y}$$

the adjoint of every value, seeded at the result, pushed back one edge at a time



Here `backward()` of `z · σ` adds: `adj[σ] += z · adj[y]`

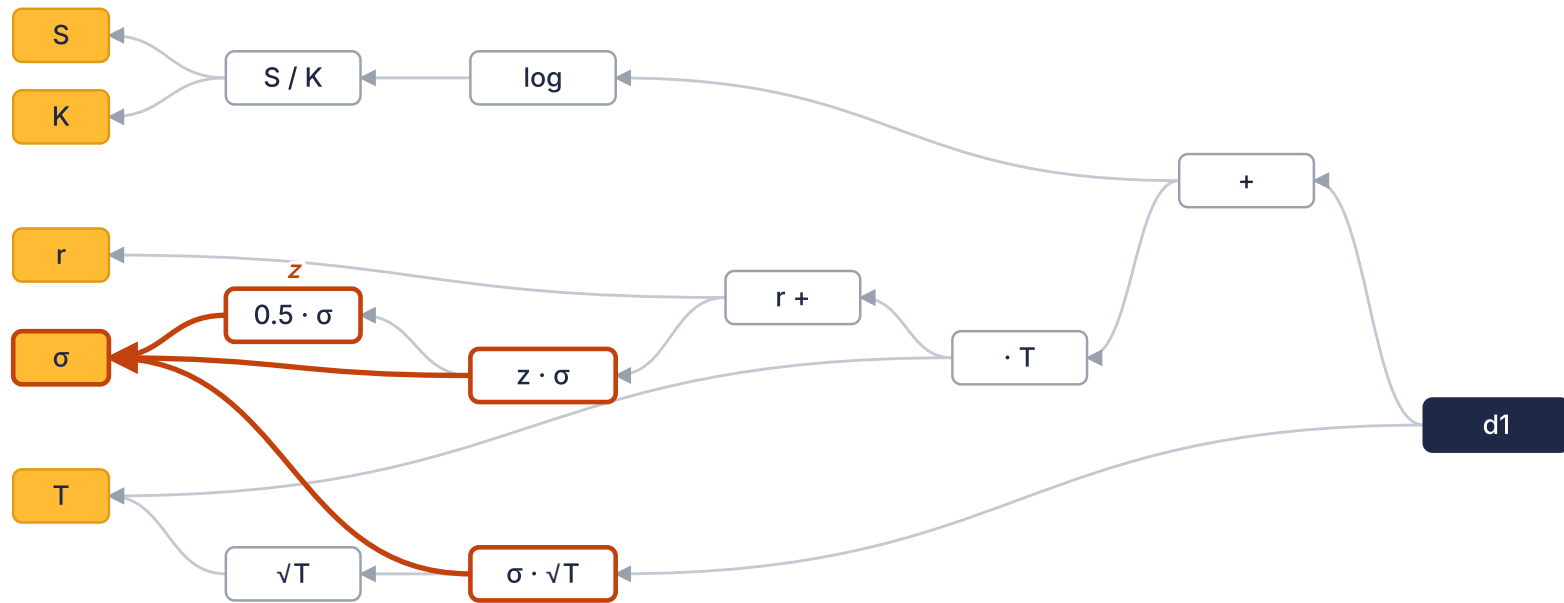
Backward sweep: chain rule

$$\bar{v} \equiv \frac{\partial d_1}{\partial v}$$

$$\overline{d_1} = 1$$

$$\bar{x} += \frac{\partial y}{\partial x} \bar{y}$$

the adjoint of every value, seeded at the result, pushed back one edge at a time



Here `backward()` of `z · σ` adds: `adj[σ] += z · adj[y]`. Three edges leave σ , so all three add into it.

The obvious implementation: a node per operation

```
struct Node
{
    virtual ~Node() = default;
    virtual void backward() = 0; // one virtual call per recorded operation
};

struct AddNode : Node // y = a + b
{
    double *a, *b, *y; // adjoint slots
    AddNode(double* a_, double* b_, double* y_) : a(a_), b(b_), y(y_) {}
    // a_b += dy/da * y_b ; b_b += dy/db * y_b;
    void backward() override { *a += *y; *b += *y; }
};

struct MulNode : Node // y = a * b
{
    double *a, *b, *y;
    double av, bv; // the operands, kept so the partial can be formed on the way back
    MulNode(double* a_, double av_, double* b_, double bv_, double* y_)
        : a(a_), b(b_), y(y_), av(av_), bv(bv_) {}
    // a_b += dy/da * y_b ; b_b += dy/db * y_b;
    void backward() override { *a += bv * *y; *b += av * *y; }
};
```

A node type per operator, full graph on tape.

The obvious implementation: a node per operation

```
struct Node
{
    virtual ~Node() = default;
    virtual void backward() = 0; // one virtual call per recorded operation
};

struct AddNode : Node // y = a + b
{
    double *a, *b, *y; // adjoint slots
    AddNode(double* a_, double* b_, double* y_) : a(a_), b(b_), y(y_) {}
    // a_b += dy/da * y_b ; b_b += dy/db * y_b;
    void backward() override { *a += *y; *b += *y; }
};

struct MulNode : Node // y = a * b
{
    double *a, *b, *y;
    double av, bv; // the operands, kept so the partial can be formed on the way back
    MulNode(double* a_, double av_, double* b_, double bv_, double* y_)
        : a(a_), b(b_), y(y_), av(av_), bv(bv_) {}
    // a_b += dy/da * y_b ; b_b += dy/db * y_b;
    void backward() override { *a += bv * *y; *b += av * *y; }
};
```

A node type per operator, full graph on tape.

The obvious implementation: a node per operation

```
struct Node
{
    virtual ~Node() = default;
    virtual void backward() = 0; // one virtual call per recorded operation
};

struct AddNode : Node // y = a + b
{
    double *a, *b, *y; // adjoint slots
    AddNode(double* a_, double* b_, double* y_) : a(a_), b(b_), y(y_) {}
    // a_b += dy/da * y_b ; b_b += dy/db * y_b;
    void backward() override { *a += *y; *b += *y; }
};

struct MulNode : Node // y = a * b
{
    double *a, *b, *y;
    double av, bv; // the operands, kept so the partial can be formed on the way back
    MulNode(double* a_, double av_, double* b_, double bv_, double* y_)
        : a(a_), b(b_), y(y_), av(av_), bv(bv_) {}
    // a_b += dy/da * y_b ; b_b += dy/db * y_b;
    void backward() override { *a += bv * *y; *b += av * *y; }
};
```

A node type per operator, full graph on tape.

Recording: an allocation per operation

```
inline Number operator*(const Number& a, const Number& b)
{
    Number r(a.value * b.value); // takes an adjoint slot from the tape ...
    g_tape->record(std::make_unique<MulNode>(a.adjoint, a.value,
                                              b.adjoint, b.value, r.adjoint));
    return r; // ... and heap-allocates a node holding both operands, every operation
}
```

One of these per operator and `d1` runs them ten times.

The sweep: pointer chase, indirect branch

```
class Tape
{
public:
    Tape() { g_tape = this; }
    ~Tape() { g_tape = nullptr; }

    double* newAdjoint()
    {
        adjoints_.push_back(0.0); // a deque, so the pointers we hand out stay
        return &adjoints_.back(); // valid, at one indirection per adjoint
    }

    void record(std::unique_ptr<Node> n) { nodes_.push_back(std::move(n)); }

    void computeAdjoint()
    {
        for (auto& node : nodes_ | std::views::reverse)
            node->backward(); // pointer chase, then an indirect branch
    }
}
```

The naive tape, measured

1000 `d1` evaluations, one tape, one reverse sweep:

	ns per d1	instructions	branches	vs primal
primal <code>double</code>	5.41	84	9	1×
naive tape	283.08	5,560	1,191	52×

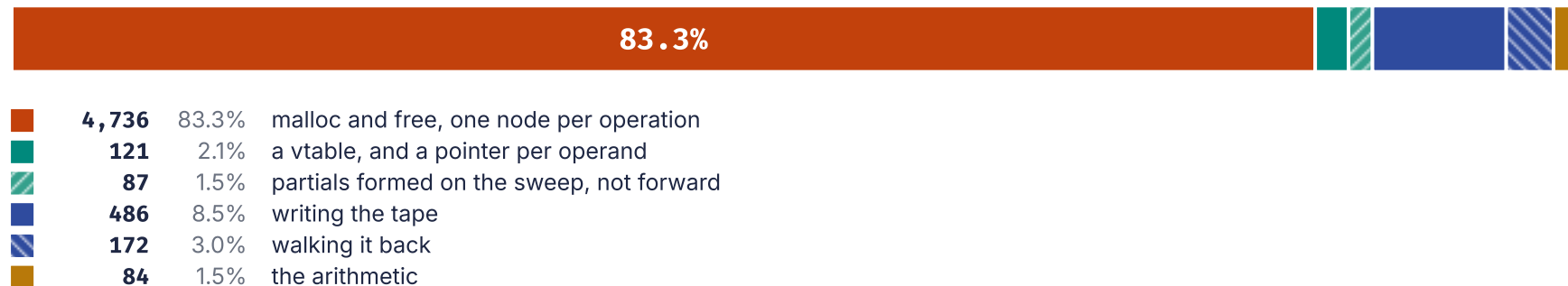
Tape cost of one `d1` statement: **10 entries, 736 bytes.**

```
$ perf stat -e instructions,branches,branch-misses ./bench_d1 --stage naive --reps 10000
55,621,649,209      instructions
11,902,745,700      branches
  7,157,908         branch-misses
```

`--reps 10000` runs those 1,000 evaluations ten thousand times: divide by 10^7 and you are within a few counts of the row above.

Where those instructions go

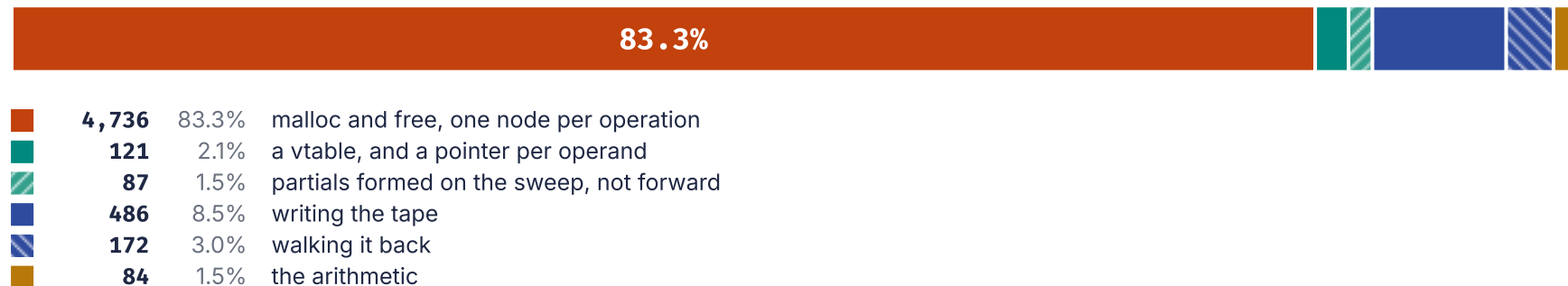
Five builds of the naive tape, one cost removed at a time:



Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · the naive ladder, 1,000 gradients of d1, averaged per d1

Where those instructions go

Five builds of the naive tape, one cost removed at a time:

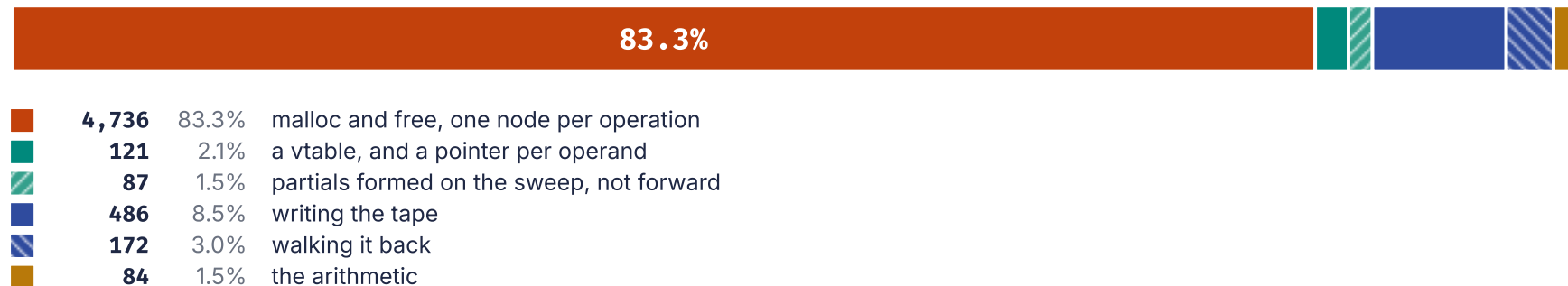


	instructions	loads	stores	stalled cycles	
a heap node per operation	5,686	1,529	870	7%	instruction-bound on <code>malloc</code>

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · the naive ladder, 1,000 gradients of d1, averaged per d1

Where those instructions go

Five builds of the naive tape, one cost removed at a time:



	instructions	loads	stores	stalled cycles	
a heap node per operation	5,686	1,529	870	7%	instruction-bound on <code>malloc</code>
flat array of operations	742	258	179	39%	waiting on its own stores

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · the naive ladder, 1,000 gradients of d1, averaged per d1

RULE 1

Never allocate per operation

83% of a naive tape's instructions are malloc and free

What do we really need for the sweep?

$$\bar{x} += \frac{\partial y}{\partial x} \bar{y}$$

Naive tape storage

- A node object and a vtable pointer
- Both operand values
- Three 8-byte pointers into the adjoint store

Really needed:

- The partial $\frac{\partial y}{\partial x}$ on each edge
- The adjoint slots for x and y

What do we really need for the sweep?

$$\bar{x} += \frac{\partial y}{\partial x} \bar{y}$$

Naive tape storage

- A node object and a vtable pointer
- Both operand values
- Three 8-byte pointers into the adjoint store

Really needed:

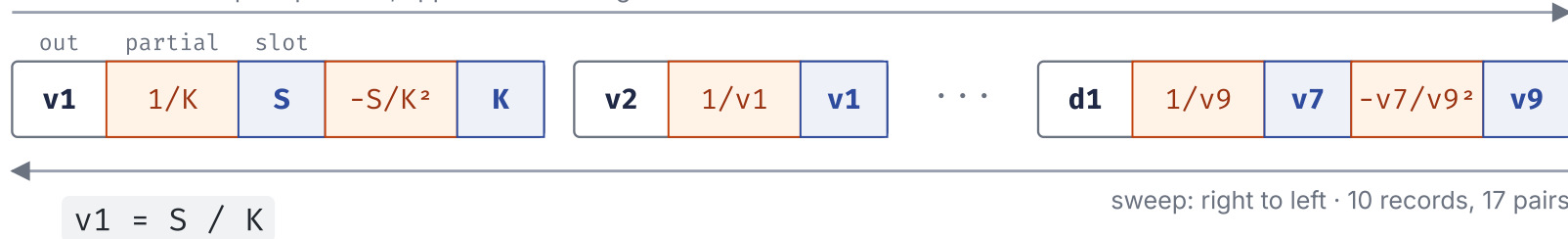
- The partial $\frac{\partial y}{\partial x}$ on each edge
- The adjoint slots for x and y

We can compute partials in forward pass while operands are still in registers and avoid storing nodes.

The design decision

A tape is a log, not a graph.

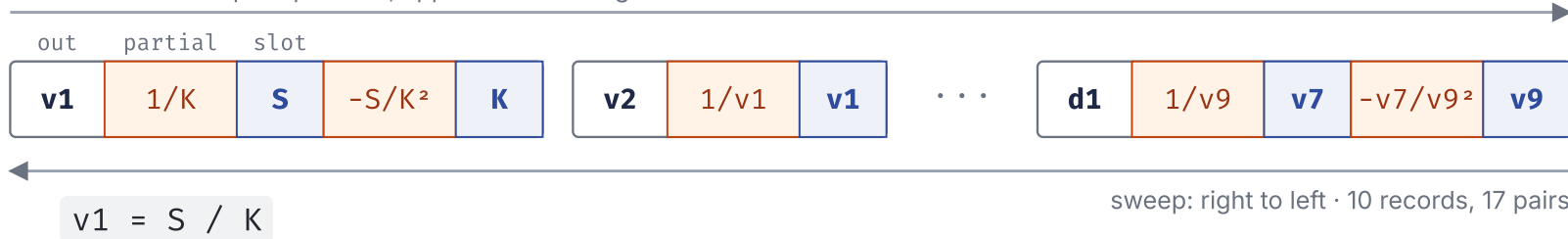
record: one record per operation, appended left to right



The design decision

A tape is a log, not a graph.

record: one record per operation, appended left to right



```
for (const auto& op : operations | std::views::reverse) // the log, backwards
{
    double a = adj[op.out]; // the slot this operation wrote
    adj[op.out] = 0.0;
    for (auto [partial, slot] : op.operands) // one or two, for now
        adj[slot] += partial * a;
}
```

No pointer chasing, virtual dispatch, or graph walk.

RULE 2

A tape is a log, not a graph

the sweep needs a partial and a slot; the operators and the values are dead weight

The rules so far

- ▶ **Never allocate per operation**

83% of a naive tape's instructions are malloc and free

- ▶ **A tape is a log, not a graph**

the sweep needs a partial and a slot; the operators and the values are dead weight

2. A chunked arena

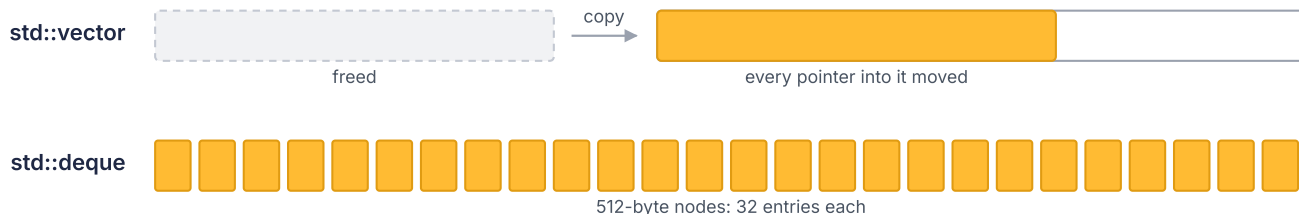
Growing a tape without moving what is already written.

Tape storage: Why not vector, why not deque?



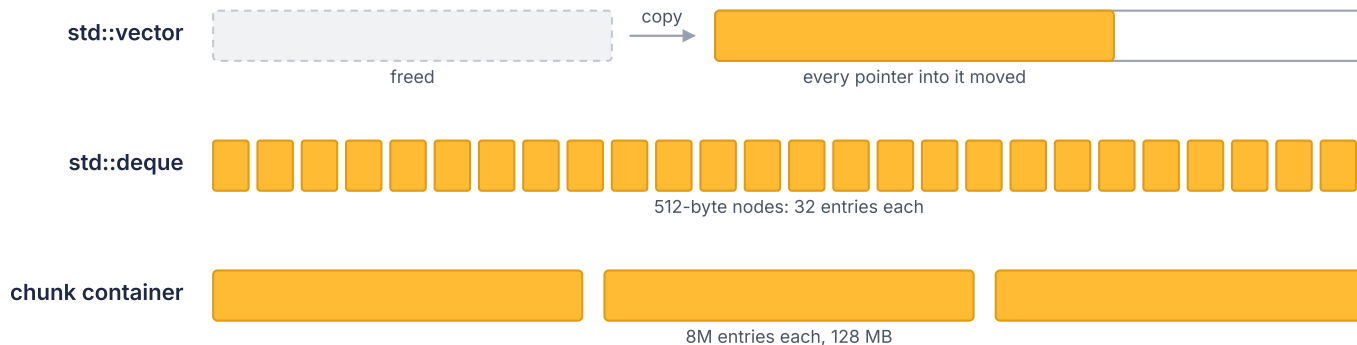
a 1 GB tape, 16 bytes per entry	blocks	addresses stable	a statement's run
<code>std::vector</code>	1, recopied on every growth	no	contiguous

Tape storage: Why not vector, why not deque?



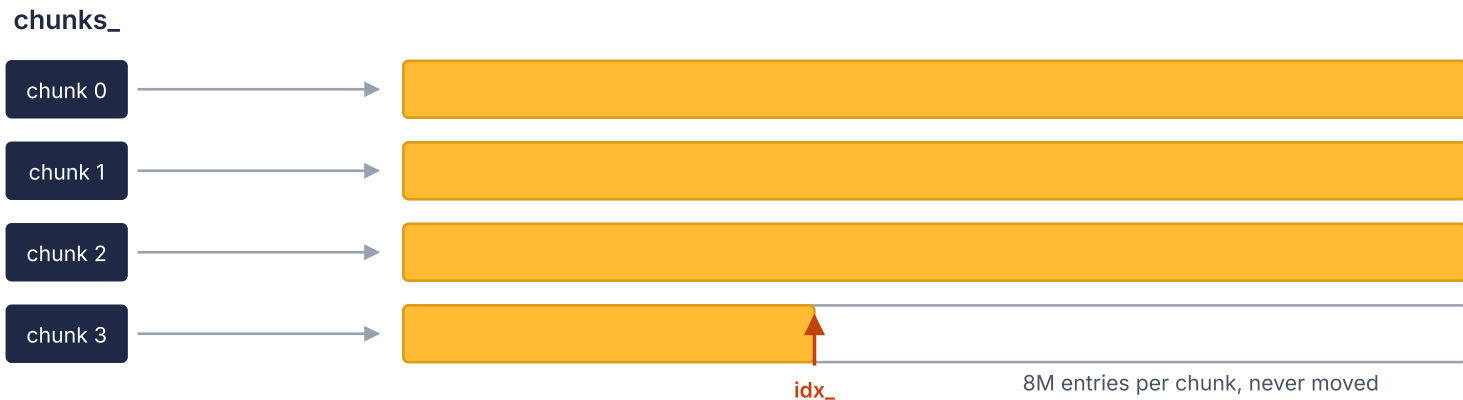
a 1 GB tape, 16 bytes per entry	blocks	addresses stable	a statement's run
<code>std::vector</code>	1, recopied on every growth	no	contiguous
<code>std::deque</code>	2,097,152	yes	split every 32 entries

Tape storage: Why not vector, why not deque?



a 1 GB tape, 16 bytes per entry	blocks	addresses stable	a statement's run
<code>std::vector</code>	1, recopied on every growth	no	contiguous
<code>std::deque</code>	2,097,152	yes	split every 32 entries
chunk container	8	yes	contiguous

The chunk container



- A `vector` of chunk pointers, room for 64 reserved up front
- 8M entries per chunk, 128 MB, aligned to 128 bytes
- Index `n` is `chunk n >> 23`, offset `n & (8M - 1)`: a shift and a mask

RULE 3

Grow in chunks, and never move what is already written

a gigabyte of tape is 8 blocks, not 2 million, and every pointer into it stays valid

Recording is a compare, a store and an increment

Branch hints: telling the compiler which way it goes

```
void push_back_unsafe(double partial, slot_type slot)
{
    if (idx_ == ChunkSize) [[unlikely]]
    {
        ++chunk_;
        idx_ = 0;
    }
    chunk(chunk_)[idx_] = {partial, slot};
    ++idx_;
}
```

`_unsafe` : the caller has already made sure that chunk exists.

Recording is a compare, a store and an increment

Branch hints: telling the compiler which way it goes

```
void push_back_unsafe(double partial, slot_type slot)
{
    if (idx_ == ChunkSize) [[unlikely]]
    {
        ++chunk_;
        idx_ = 0;
    }
    chunk(chunk_)[idx_] = {partial, slot};
    ++idx_;
}
```

`_unsafe` : the caller has already made sure that chunk exists.

Recording is a compare, a store and an increment

Branch hints: telling the compiler which way it goes

```
void push_back_unsafe(double partial, slot_type slot)
{
    if (idx_ == ChunkSize) [[unlikely]]
    {
        ++chunk_;
        idx_ = 0;
    }
    chunk(chunk_)[idx_] = {partial, slot};
    ++idx_;
}
```

`_unsafe` : the caller has already made sure that chunk exists.

Recording is a compare, a store and an increment

Branch hints: telling the compiler which way it goes

```
void push_back_unsafe(double partial, slot_type slot)
{
    if (idx_ == ChunkSize) [[unlikely]]
    {
        ++chunk_;
        idx_ = 0;
    }
    chunk(chunk_)[idx_] = {partial, slot};
    ++idx_;
}
```

[[unlikely]] (C++20) moves that block out of the hot path.

→ the hot path is straight-line code with nothing to jump over

_unsafe : the caller has already made sure that chunk exists.

A whole statement in one append

```
void append_n(const std::pair<double, slot_type>* run, std::size_t n)
{
    auto items = std::min(ChunkSize - idx_, n);
    std::copy_n(run, items, chunk(chunk_) + idx_);
    idx_ += items;

    if (idx_ == ChunkSize) [[unlikely]] // the run straddles a chunk
    {
        next_chunk();
        append_n(run + items, n - items); // the rest, from the top
    }
}
```

- `append_n` on the `(partial, slot)` container: the boundary check happens **once per statement**
- The `if` only fires when a statement straddles a chunk

The branch that never misses

```
$ perf stat -e instructions,branches,branch-misses ./bench_d1 --stage <variant> --reps 10000
```

primal

803,746,501	instructions
90,674,537	branches
26,933	branch-misses

naive tape

55,621,649,209	instructions
11,902,745,700	branches
7,157,908	branch-misses

XAD

6,536,960,152	instructions
782,621,242	branches
52,139	branch-misses

Ubuntu 26.04 · GCC 15.2 · -O3 -march=cascadelake · Xeon Platinum 8488C, one core · 10,000 loops of 1,000 d1 gradients

The branch that never misses

```
$ perf stat -e instructions,branches,branch-misses ./bench_d1 --stage <variant> --reps 10000
```

primal

```
803,746,501    instructions
 90,674,537    branches
 26,933        branch-misses
```

naive tape

```
55,621,649,209 instructions
11,902,745,700  branches
 7,157,908      branch-misses
```

XAD

```
6,536,960,152  instructions
 782,621,242    branches
 52,139          branch-misses
```

Less what starting the program costs (`--reps 0`), divided by ten million, one per `d1` gradient:

	branches executed	of those, mispredicted	miss rate
primal	9	0.001	0.01%
naive tape	1,190	0.71	0.06%
XAD	78	0.004	0.005%

- One miss every 270 gradients
- A chunk holds 8M entries, and `append_n` asks once per statement

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · 10,000 loops of 1,000 `d1` gradients

RULE 4

Check the chunk boundary once per statement and help the branch predictor

78 branches per gradient, mispredicted 0.005% of the time

The rules so far

- ✓ Never allocate per operation

83% of a naive tape's instructions are malloc and free

- ✓ A tape is a log, not a graph

the sweep needs a partial and a slot; the operators and the values are dead weight

- ▶ **Grow in chunks, and never move what is already written**

a gigabyte of tape is 8 blocks, not 2 million, and every pointer into it stays valid

- ▶ **Check the chunk boundary once per statement and help the branch predictor**

78 branches per gradient, mispredicted 0.005% of the time

3. Expression templates

The whole right-hand side becomes one statement.

Plain overloads: every operation is a variable

```
Real a, b, c;  
Real y = a * b + c;    // t1 = a*b    → a slot, a statement  
                        // y = t1+c    → a slot, a statement
```

Plain overloads: every operation is a variable

```
Real a, b, c;  
Real y = a * b + c;      // t1 = a*b   → a slot, a statement  
                        // y = t1+c   → a slot, a statement
```

- d1 's ten operations become **21 statements, 28 operations**
- Every temporary gets a slot, every assignment to one is a statement, and each result is copied out of its temporary

The sum is a type

```
template <class Derived>
struct Expression
{
    const Derived& derived() const { return static_cast<const Derived&>(*this); }
    double value() const { return derived().value(); }

    template <class Info>
    void calc_derivatives(Info& info, double mul) const
    {
        derived().calc_derivatives(info, mul);
    }
};
```

- CRTP: each node knows its own type, so the compiler sees the whole tree
- Nothing reaches the tape while the expression is built

What `d1` becomes

```
d1 = (log(S / K) + (r + 0.5 * sigma * sigma) * T) / (sigma * sqrt(T));
```

What `d1` becomes

```
d1 = (log(S / K) + (r + 0.5 * sigma * sigma) * T) / (sigma * sqrt(T));
```

```
BinaryExpr<DivOp,  
  BinaryExpr<AddOp,  
    UnaryExpr<LogOp, BinaryExpr<DivOp, Variable, Variable>>,  
    BinaryExpr<MulOp,  
      BinaryExpr<AddOp, Variable,  
        BinaryExpr<MulOp, UnaryExpr<ScalarMulOp, Variable>, Variable>>,  
        Variable>>,  
      BinaryExpr<MulOp, Variable, UnaryExpr<SqrtOp, Variable>>>
```

What `d1` becomes

```
d1 = (log(S / K) + (r + 0.5 * sigma * sigma) * T) / (sigma * sqrt(T));
```

```
BinaryExpr<DivOp,  
  BinaryExpr<AddOp,  
    UnaryExpr<LogOp, BinaryExpr<DivOp, Variable, Variable>>,  
    BinaryExpr<MulOp,  
      BinaryExpr<AddOp, Variable,  
        BinaryExpr<MulOp, UnaryExpr<ScalarMulOp, Variable>, Variable>>,  
        Variable>>,  
      BinaryExpr<MulOp, Variable, UnaryExpr<SqrtOp, Variable>>>
```

Eight leaves: **one statement, eight partials**. `0.5` is not one of them: a constant times a variable is a one-operand node.

Expression is evaluated on assignment to `d1`.

The buffer is sized by the type

```
template <int N>
struct DerivInfo
{
    unsigned index = 0;
    std::pair<double, slot_type> partials[N];
};
```

```
numVariables = ExprTraits<L>::numVariables + ExprTraits<R>::numVariables
```

Sized at compile time from the expression's type.

A stack array. No allocation, no bookkeeping.

A leaf contributes one partial

```
template <class Info>
void calc_derivatives(Info& info, double partial) const
{
    info.partials[info.index++] = {partial, slot_};
}
```

A leaf contributes one `(partial, slot)` pair, carrying the accumulated chain-rule multiplier on the way down.

Each node scales the multiplier

```
template <class Info>
void calc_derivatives(Info& info, double mul) const
{
    left.calc_derivatives(info, mul * op.d_left(left.value(), right.value(), val_));
    right.calc_derivatives(info, mul * op.d_right(left.value(), right.value(), val_));
}
```

- The chain rule as a recursive descent: each node scales `mul` by its own partial and hands it to both sides
- All of it inlines, so what is left is straight-line arithmetic

One append, one statement

The only place anything reaches the tape

```
template <class E>
Variable& Variable::operator=(const Expression<E>& expr)
{
    DerivInfo<ExprTraits<E>::numVariables> info;
    expr.derived().calc_derivatives(info, 1.0);

    val_ = expr.value();
    slot_ = g_tape->newSlot();
    g_tape->operations.append_n(info.partials, info.index);
    g_tape->statements.push_back({g_tape->operations.size(), slot_});
    return *this;
}
```

The whole `d1` line: **one statement, eight partials.**

One append, one statement

The only place anything reaches the tape

```
template <class E>
Variable& Variable::operator=(const Expression<E>& expr)
{
    DerivInfo<ExprTraits<E>::numVariables> info;
    expr.derived().calc_derivatives(info, 1.0);

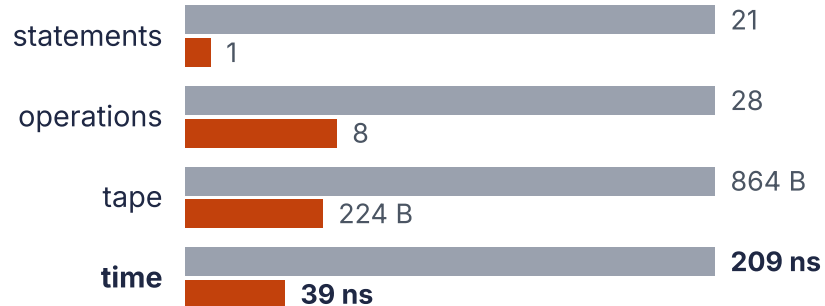
    val_ = expr.value();
    slot_ = g_tape->newSlot();
    g_tape->operations.append_n(info.partials, info.index);
    g_tape->statements.push_back({g_tape->operations.size(), slot_});
    return *this;
}
```

The whole `d1` line: **one statement, eight partials.**

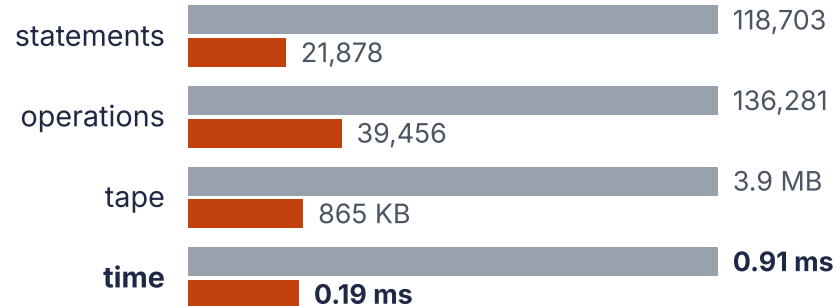
What it is worth

■ every operation materialises = 100% ■ the right-hand side is a type

`d1` , one statement



CVA, one path



81% and 79% less time, on 74% and 78% less tape.

- The naive tape's ten nodes were 736 B: the arena did not shrink the tape, it stopped allocating
- 39 ns is **7.2×** the primal — one statement leaves almost nothing to amortise; the four workloads are 2.9× to 6.7×

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · XAD; one `d1` statement, one CVA path

RULE 5

The right-hand side is a type

one statement instead of 21, and a quarter of the tape

The rules so far

- ✓ Never allocate per operation

83% of a naive tape's instructions are malloc and free

- ✓ A tape is a log, not a graph

the sweep needs a partial and a slot; the operators and the values are dead weight

- ✓ Grow in chunks, and never move what is already written

a gigabyte of tape is 8 blocks, not 2 million, and every pointer into it stays valid

- ✓ Check the chunk boundary once per statement and help the branch predictor

78 branches per gradient, mispredicted 0.005% of the time

- ▶ **The right-hand side is a type**

one statement instead of 21, and a quarter of the tape

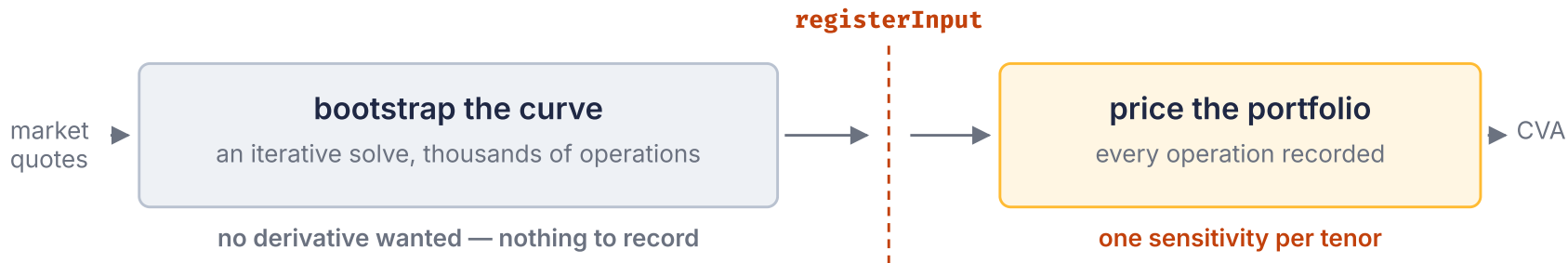
4. shouldRecord, and what is left

Asking before recording, writing once, and the smaller trades.

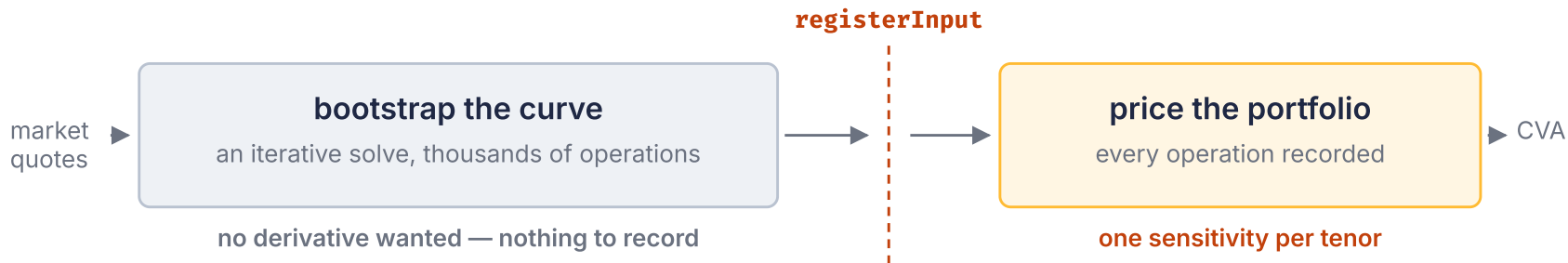
The four measured kernels for this section

- **Heston MC:**
Monte-Carlo option pricer using the Heston stochastic volatility model, 8 inputs
- **SABR calibration:**
Calibration of the SABR stochastic volatility model, 15 inputs
- **CVA:**
Monte-Carlo credit valuation adjustment, 40 inputs
- **LIBOR swaption:**
Monte-Carlo swaption portfolio pricer under LIBOR market model, 161 inputs

Most arithmetic is not the arithmetic you want

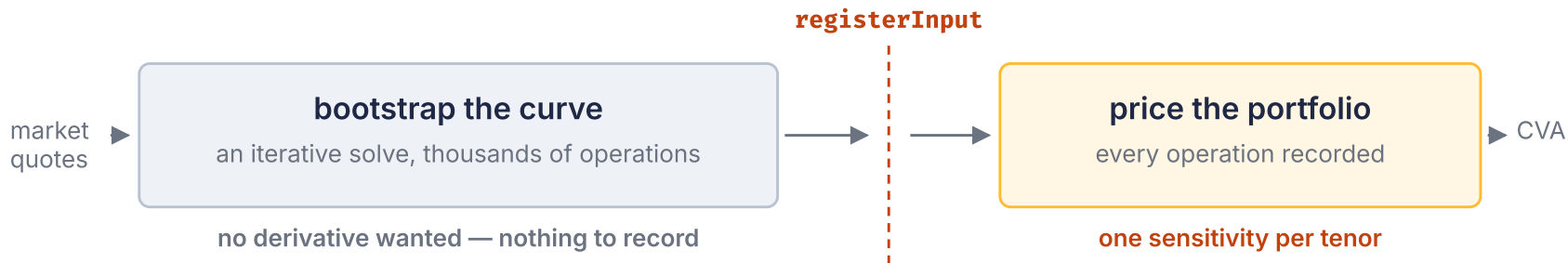


Most arithmetic is not the arithmetic you want



- A value is active only if it **descends from a registered input**
- `shouldRecord()` asks the expression tree, statement by statement
- Same `Real` type either side; only one side reaches the tape

Most arithmetic is not the arithmetic you want



- A value is active only if it **descends from a registered input**
- `shouldRecord()` asks the expression tree, statement by statement
- Same `Real` type either side; only one side reaches the tape

Asking has to cost nothing, or you pay it on everything.

The order of two cheap checks

Ask the operands first:

```
if (expr.shouldRecord())           // walk the operands
{
    auto s = tape_type::getActive(); // then find the tape
    ...
}
```

Ask the tape first:

```
auto s = tape_type::getActive(); // one load, short-circuits
if (s && expr.shouldRecord())
{
    ...
}
```

With no tape at all, this still has to ask

```
Real a = 3.0, b; // the active type, and no tape has ever been created
b = a;           // does this need recording?
```

flag first: two loads, two compares, two branches

```
mov    0x8(%rsi),%esi
cmp    $0xffffffff,%esi
jne    <record>
cmpl   $0xffffffff,0x8(%rdi)
jne    <record>
```

tape first: one load, one test, one branch

```
mov    $0xfffffffffffffffff0,%rax
mov    %fs:(%rax),%rcx
test   %rcx,%rcx
je     <nothing to do>
```

- Two operands here, so two slot checks
- The `d1` expression has **eight leaves**: eight checks, still one tape load

With no tape at all, this still has to ask

```
Real a = 3.0, b; // the active type, and no tape has ever been created
b = a;           // does this need recording?
```

flag first: two loads, two compares, two branches

```
mov    0x8(%rsi),%esi
cmp     $0xffffffff,%esi
jne     <record>
cmpl    $0xffffffff,0x8(%rdi)
jne     <record>
```

tape first: one load, one test, one branch

```
mov     $0xfffffffffffffffff0,%rax
mov     %fs:(%rax),%rcx
test    %rcx,%rcx
je      <nothing to do>
```

- Two operands here, so two slot checks
- The `d1` expression has **eight leaves**: eight checks, still one tape load
- **-5.6%** instructions on `d1` with no tape active
- **-0.04%** on the CVA gradient: it records almost everything

Making the question cheap

$$d_1 = \frac{\log(S/K) + (r + \frac{1}{2} \sigma \sigma) T}{\sigma \sqrt{T}}$$

- Eight leaves, **five variables**: three σ , two T
- Inlined, the compiler folds the repeats; not inlined, it asks all eight → force-inline it

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · XAD; 3 interleaved runs; the bars, minimum of 3

Making the question cheap

$$d_1 = \frac{\log(S/K) + (r + \frac{1}{2} \sigma \sigma) T}{\sigma \sqrt{T}}$$

- Eight leaves, **five variables**: three σ , two T
- Inlined, the compiler folds the repeats; not inlined, it asks all eight → force-inline it

Not inlining `shouldRecord()` costs:



Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · XAD; 3 interleaved runs; the bars, minimum of 3

RULE 6

**Ask whether to record
before recording and
make the question cheap**

the largest single effect on real code

Write the partials once

Every statement built its partials on the stack, then copied them to the tape.

```
DerivInfo<N> info;                                // stack buffer
expr.derived().calc_derivatives(info, 1.0);        // partials written here
g_tape→operations.append_n(info.partials, info.index); // ...then copied
```

→ Extra copy per operation

Write the partials once

Every statement built its partials on the stack, then copied them to the tape.

```
DerivInfo<N> info;                                // stack buffer
expr.derived().calc_derivatives(info, 1.0);        // partials written here
g_tape→operations.append_n(info.partials, info.index); // ...then copied
```

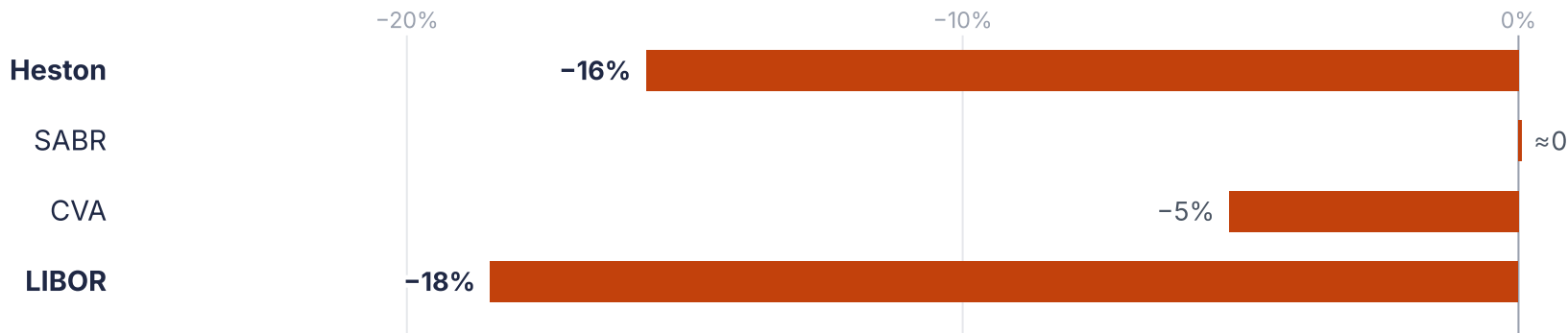
→ Extra copy per operation

N is known at compile-time from expression, so reserve first and write in place:

```
auto* dst = g_tape→operations.try_reserve(N); // null at a chunk boundary
if (!dst) [[unlikely]]
    return *this = expr;                       // the staged path, unchanged
DirectInfo info{dst};                          // no stack buffer
expr.derived().calc_derivatives(info, 1.0);    // straight into the tape
g_tape→operations.commit(info.index);
```

What writing once was worth

Change in gradient time from writing straight into the tape, same build otherwise:



Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · XAD, minimum of three interleaved repetitions

RULE 7

Write the partials straight into the tape

the staging buffer was costing more than the scattered stores saved

Reuse the result you already have

```
template <class Scalar>
struct exp_op
{
    Scalar operator()(const Scalar& a) const { return exp(a); }

    // v is the forward result: exp(a), computed a moment ago
    Scalar derivative(const Scalar&, const Scalar& v) const { return v; }
};

template <class Scalar>
struct OperatorTraits<exp_op<Scalar>> { static constexpr bool useResultBasedDerivatives = true; };
```

$\frac{d}{dx}e^x = e^x$: the forward pass already has it in a register.

Reuse the result you already have

```
template <class Scalar>
struct exp_op
{
    Scalar operator()(const Scalar& a) const { return exp(a); }

    // v is the forward result: exp(a), computed a moment ago
    Scalar derivative(const Scalar&, const Scalar& v) const { return v; }
};

template <class Scalar>
struct OperatorTraits<exp_op<Scalar>> { static constexpr bool useResultBasedDerivatives = true; };
```

$\frac{d}{dx}e^x = e^x$: the forward pass already has it in a register.

Recomputing it instead costs:



Worth what the kernel spends in `exp`, `sqrt` and `pow`, and no more.

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · XAD, minimum of three repetitions

RULE 8

Reuse the result you already have

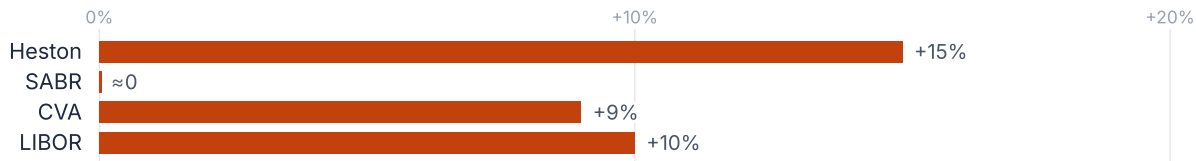
recomputing a partial the forward pass already had costs LIBOR 11% and CVA 7%

Skip the sweep when the adjoint is zero

```
for (const auto& stmt : statements | std::views::reverse)
{
    double a = adj[stmt.out];
    adj[stmt.out] = 0.0;
    if (a == 0.0) continue; // nothing to propagate
    for (auto [partial, slot] : stmt.entries)
        adj[slot] += partial * a;
}
```

A zero adjoint contributes nothing, so the whole statement is skipped.

Extra gradient time with the skip switched off:



Ubuntu 26.04 · GCC 15.2 · -O3 -march=cascadelake · Xeon Platinum 8488C, one core · XAD, minimum of three repetitions

RULE 9

Skip the sweep when the adjoint is zero

without it, three kernels run 9–15% slower, and SABR does not notice

Two ways to trade time for tape

	LIBOR, tape cleared each path		CVA risk run, one tape	
	tape	gradient time	tape	gradient time
default	553 KB	1×	885 MB	1×
reuse dead slots	-0%	+41%	-9%	+28%

A dead slot goes back on a free list: worth something only when the tape outlives the path.

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · XAD build options; one LIBOR gradient; one CVA risk run, 1,000 paths

Two ways to trade time for tape

	LIBOR, tape cleared each path		CVA risk run, one tape	
	tape	gradient time	tape	gradient time
default	553 KB	1×	885 MB	1×
reuse dead slots	-0%	+41%	-9%	+28%
compact storage	-22%	+43%	-18%	-6%

A dead slot goes back on a free list: worth something only when the tape outlives the path.



Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · XAD build options; one LIBOR gradient; one CVA risk run, 1,000 paths

Two ways to trade time for tape

	LIBOR, tape cleared each path		CVA risk run, one tape	
	tape	gradient time	tape	gradient time
default	553 KB	1×	885 MB	1×
reuse dead slots	-0%	+41%	-9%	+28%
compact storage	-22%	+43%	-18%	-6%
both	-22%	+87%	-27%	+25%

A dead slot goes back on a free list: worth something only when the tape outlives the path.



Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · XAD build options; one LIBOR gradient; one CVA risk run, 1,000 paths

The rules so far

- ✓ Never allocate per operation
 - ✓ A tape is a log, not a graph
 - ✓ Grow in chunks, and never move what is already written
 - ✓ Check the chunk boundary once per statement and help the branch predictor
 - ✓ The right-hand side is a type
- ▶ **Ask whether to record before recording and make the question cheap**
 - ▶ **Write the partials straight into the tape**
 - ▶ **Reuse the result you already have**
 - ▶ **Skip the sweep when the adjoint is zero**

5. Where that leaves us



The numbers and the rules.

CVA, 10,000 paths: from 55x to 3.9x

Gradient time against the primal, each design in a binary of its own:



Ubuntu 26.04 · GCC 15.2 · -O3 -march=cascadelake · Xeon Platinum 8488C, one core · median of 5 interleaved repetitions of 10 runs, one binary per bar

CVA, 10,000 paths: from 55x to 3.9x

Gradient time against the primal, each design in a binary of its own:



Zoomed in: expression templates alone leave CVA above the bound, rules 6 to 9 take it under.

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · median of 5 interleaved repetitions of 10 runs, one binary per bar

What each optimisation is worth

Extra gradient time with each one switched off:

	Heston	SABR	CVA	LIBOR
shouldRecord() not inlined	+29%	+161%	+8%	+116%
constants recorded as operands		+27%		
no branch hints	+15%		+22%	+9%
no sweep skip	+15%		+9%	+10%
no result reuse			+7%	+11%

- Blank is under 5%, inside these benchmarks' repeat spread
- **Four of five rows** are double digits on one kernel and nothing on another

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · XAD, minimum of three repetitions

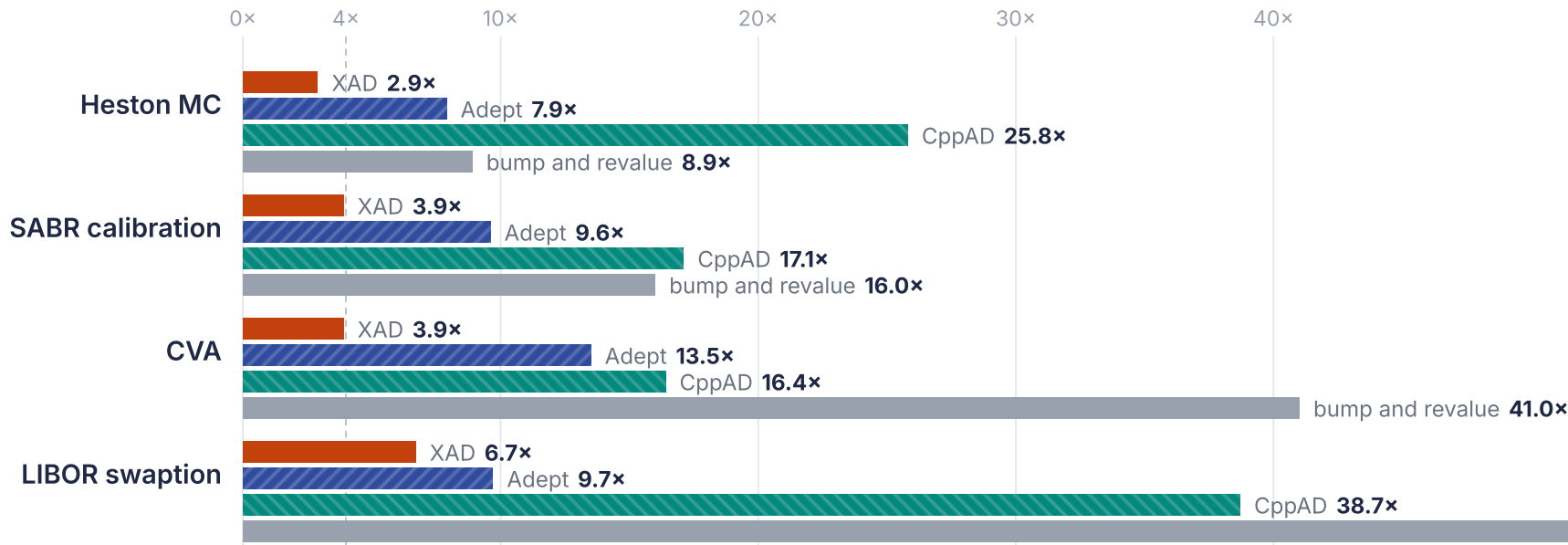
RULE 10

One kernel will lie to you

the workloads disagree about the ranking

Final performance

Overhead over the undifferentiated primal, compared to alternative AD tools



Bump and revalue runs off the slide on LIBOR: 162 x

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · one configuration, each library as documented

Why LIBOR is 6.7× and Heston 2.9×

Clock cycles and instructions per operation XAD records, and what the primal spent on it:

per operation	primal cycles	primal instructions	primal IPC	XAD adds cycles	XAD adds instructions
Heston MC	1.9	1.2	0.6	4.1	32
LIBOR swaption	1.2	4.9	4.1	6.8	33

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · per operation recorded, one path per workload

Why LIBOR is 6.7× and Heston 2.9×

Clock cycles and instructions per operation XAD records, and what the primal spent on it:

per operation	primal cycles	primal instructions	primal IPC	XAD adds cycles	XAD adds instructions
Heston MC	1.9	1.2	0.6	4.1	32
LIBOR swaption	1.2	4.9	4.1	6.8	33

- LIBOR's primal has no headroom: **4 instructions a cycle**, divider busy 54%
- Heston's waits on divides and `exp`, so the tape work hides in its shadow

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · per operation recorded, one path per workload

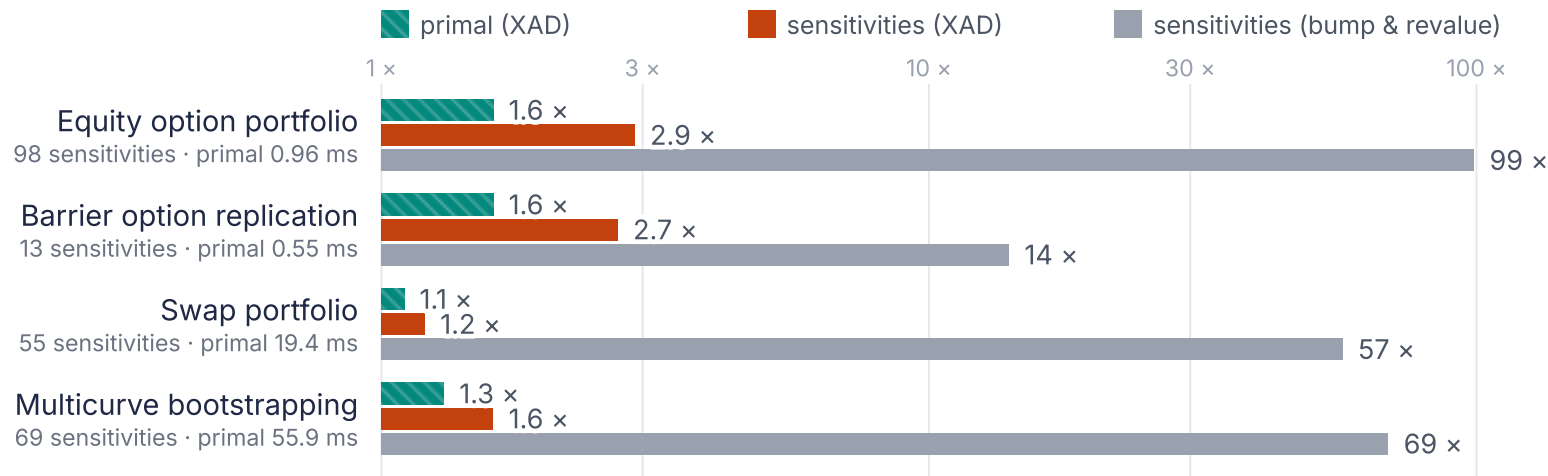
XAD in a large quant library

- QuantLib is the open example: 300,000 lines, `Real` bound to XAD's active type.
- QuantLib original code, nothing rewritten or specialised

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · QuantLib 1.43, four of its examples; median of 3 runs

XAD in a large quant library

- QuantLib is the open example: 300,000 lines, `Real` bound to XAD's active type.
- QuantLib original code, nothing rewritten or specialised



- 1.1 - 1.6x of that is the active type alone without recording
- The integration is public: github.com/auto-differentiation/QuantLibAAD

Ubuntu 26.04 · GCC 15.2 · `-O3 -march=cascadelake` · Xeon Platinum 8488C, one core · QuantLib 1.43, four of its examples; median of 3 runs

Ten rules for a fast tape

- ✓ Never allocate per operation
- ✓ A tape is a log, not a graph
- ✓ Grow in chunks, and never move what is already written
- ✓ Check the chunk boundary once per statement and help the branch predictor
- ✓ The right-hand side is a type
- ✓ Ask whether to record before recording and make the question cheap
- ✓ Write the partials straight into the tape
- ✓ Reuse the result you already have
- ✓ Skip the sweep when the adjoint is zero
- ✓ One kernel will lie to you

**55× became 3.9× by making
each tape entry cheap, then
cutting over two thirds of them.**

Thank you. Questions?



github.com/auto-differentiation/xad
github.com/auto-differentiation/QuantLibAAD